

MF0492_3. Programación web en el entorno servidor.

UF1846. Desarrollo de aplicaciones web distribuidas.

EXAMEN INTERMEDIO.

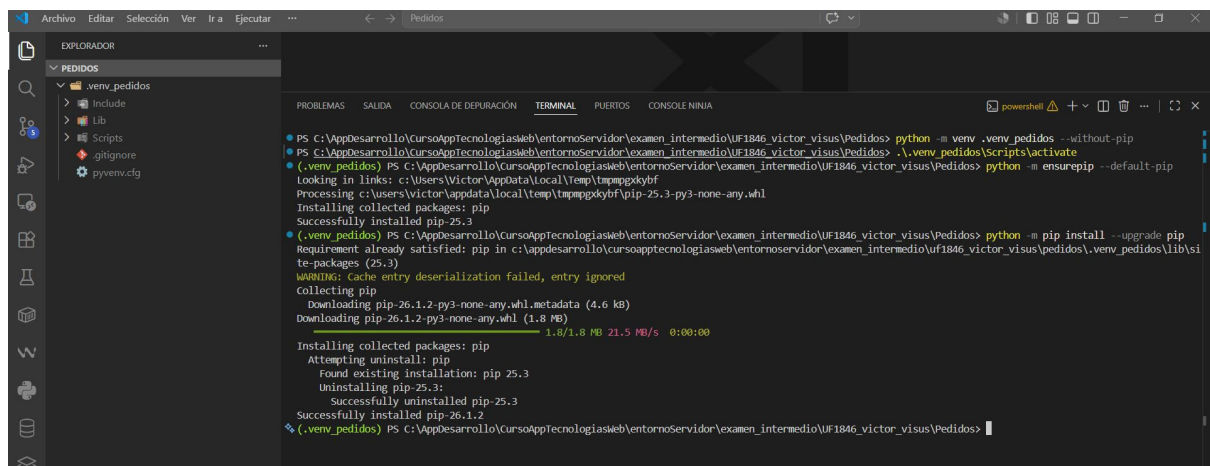
NOMBRE y APELLIDOS: **VICTOR VISÚS GARCÍA**

FECHA: **01 de JUNIO de 2026**

Para la realización de este examen se aportará un fichero con el nombre UF1846_Nombre_Apellido.pdf con las oportunas capturas de pantalla, además de un .ZIP sólo con las carpetas del proyecto (sin las dependencias virtualizadas) y el fichero requirements.txt final adecuado para la reproducción de lo codificado.

En el desarrollo que viene a continuación se asume que existe una instancia de servidor MySQL o MariaDB a la escucha por el puerto usual 3306. Se recomienda la comprobación de lo ejecutado contra el servidor con la apertura de un cliente de BD paralelo e independiente.

1. Crear un proyecto virtual para Python llamado *pedidos*. Activar el proyecto. (1 punto).



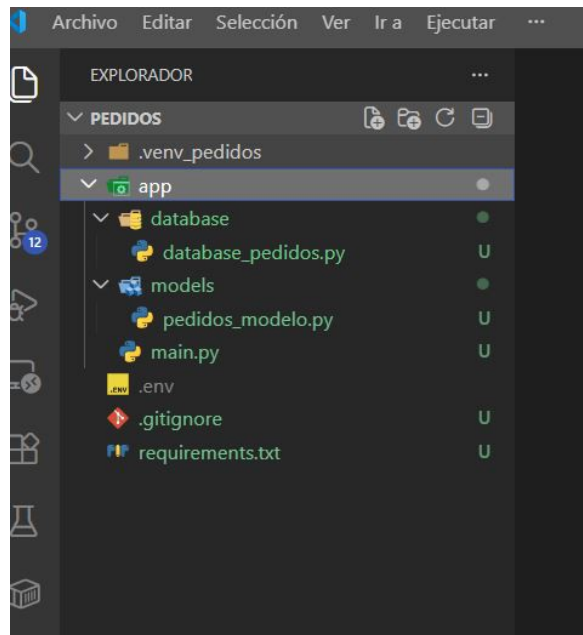
```
PS C:\AppDesarrollo\CursoAppTecnologiasWeb\entornoServidor\examen_intermedio\UF1846_victor_visus\pedidos> python -m venv .venv_pedidos --without-pip
PS C:\AppDesarrollo\CursoAppTecnologiasWeb\entornoServidor\examen_intermedio\UF1846_victor_visus\pedidos> .\.venv_pedidos\Scripts\activate
(.venv_pedidos) PS C:\AppDesarrollo\CursoAppTecnologiasWeb\entornoServidor\examen_intermedio\UF1846_victor_visus\pedidos> python -m ensurepip --default-pip
Looking in links: c:\Users\Victor\AppData\Local\Temp\tmpmpgkdybf
Processing c:\Users\Victor\AppData\Local\Temp\tmpmpgkdybf\pip-25.3-py3-none-any.whl
Installing collected packages: pip
Successfully installed pip-25.3
(.venv_pedidos) PS C:\AppDesarrollo\CursoAppTecnologiasWeb\entornoServidor\examen_intermedio\UF1846_victor_visus\pedidos> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\appdesarrollo\cursoapptecnologiasweb\entornoServidor\examen_intermedio\uf1846_victor_visus\pedidos\.venv_pedidos\lib\site-packages (25.3)
WARNING: Cache entry deserialization failed, entry ignored
Collecting pip
  Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 21.5 MB/s 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.3
    Uninstalling pip-25.3:
      Successfully uninstalled pip-25.3
  Successfully installed pip-26.1.2
(.venv_pedidos) PS C:\AppDesarrollo\CursoAppTecnologiasWeb\entornoServidor\examen_intermedio\UF1846_victor_visus\pedidos>
```

NOTA: Al crear el entorno con el comando “python -m venv .venv_pedidos” me ha lanzado un error, lo instalo “sin pip”

Los comandos usados han sido:

1. **Crear el entorno:** `python -m venv .venv_pedidos --without-pip`
2. **Activar el entorno:** `.\.venv_pedidos\Scripts\activate`
3. **Instalar pip de forma segura:** `python -m ensurepip --default-pip`
4. **Actualizar pip:** `python -m pip install --upgrade pip`

2. Crear una estructura de directorio de proyecto que incluya carpetas para: la aplicación principal (app.py o main.py), para el código de conexión/creación de base de datos y tablas (bddd.py), y también para el modelo. (1 punto).



3. Adaptándose a la estructura de 2. y haciendo uso del driver *pymysql* crear una base de datos (CON EJECUCIÓN DESDE APLICACIÓN Python, Y NO DESDE CLIENTE CONECTADO AL SGBD) de nombre *gestión_pedido* usuario root y contraseña DE LIBRE ELECCIÓN en localhost por el puerto 3306 bajo el uso de SQLAlchemy. Añadir y hacer accesible la información de conexión que deba permanecer oculta en el archivo .env convenientemente invocado.

Crear el fichero requirements.txt adecuado. (2 puntos)

FICHERO: database_pedidos.py

```
import os

from typing import Annotated

import pymysql
from dotenv import load_dotenv
from fastapi import Depends
from sqlalchemy import Session, SQLModel, create_engine

# Cargar variables de entorno desde el archivo .env
load_dotenv()

# =====
# CONFIGURACIÓN DE LA BASE DE DATOS (Estilo Clase)
# =====

USER = os.getenv("USER_DB", "root")
PASSWORD = os.getenv("PASSWORD_DB", "")
```

```

HOST = os.getenv("HOST_DB", "localhost")
PORT: int = int(os.getenv("PORT", "3306"))
NAME_DB = os.getenv("NAME_DB", "gestión_pedido")

# Construir URL de conexión a MySQL usando el driver pymysql
DATABASE_URL = f"mysql+pymysql://{USER}:{PASSWORD}@{HOST}:{PORT}/{NAME_DB}"
engine = create_engine(DATABASE_URL)

# =====
# INICIALIZACIÓN (Llamada desde el lifespan de main.py)
# =====

def inicializar_base_de_datos():
    # PASO 1: Conectarse al servidor sin especificar BD para crearla si no
    existe
    conexion_servidor = pymysql.connect(
        host=HOST, user=USER, password=PASSWORD, port=int(PORT)
    )
    try:
        with conexion_servidor.cursor() as cursor:
            # Creamos la base de datos de manera segura con acentos/eñes soportados
            cursor.execute(
                f"CREATE DATABASE IF NOT EXISTS `{NAME_DB}` CHARACTER SET utf8mb4;"
            )
    finally:
        conexion_servidor.close()

# PASO 2: Creación de tablas mediante el motor de SQLAlchemy
# IMPORTANTE: main.py debe importar los modelos antes para que se creen aquí
SQLModel.metadata.create_all(engine)

# =====
# INYECCIÓN DE DEPENDENCIAS (Necesario para los Endpoints del examen)
# =====

def get_session():
    with Session(engine) as session:
        yield session

# Tipo anotado para usarlo limpiamente en los parámetros de tus controladores
session_dep = Annotated[Session, Depends(get_session)]

```

FICHERO: .env

USER_DB=root

PASSWORD_DB=root

HOST_DB=localhost

PORT=3306

NAME_DB=gestion_pedido

4. Dados estos dos diccionario de datos (“plantilla para crear un modelo”):

Cliente:

Campo	Tipo de Dato (SQL)	PK / FK	Nullable
NIF	`VARCHAR(9)`	PK	No
nombre	`VARCHAR(50)`		No
apellidos	`VARCHAR(50)`		No
direccion	`VARCHAR(100)`		Sí

Pedido:

Campo	Tipo de Dato (SQL)	PK / FK	Nullable
id_pedido	`INT`	PK	No Autoincremental.
NIF_cliente	`VARCHAR(9)`	FK	No Ref. `NIF` de `Cliente`.
importe	`DECIMAL(10, 2)`		Sí

Crear el modelo ORM en el fichero conveniente para las entidades propuestas y ejecutar su creación como tablas, usar los atributos, modificadores y validadores que se estimen oportunos de SQLAlchemy. (3 puntos).

FICHERO: pedidos_modelo.py

```
from decimal import Decimal
```

```
from typing import Optional
```

```
from pydantic import field_validator
```

```
from sqlalchemy import Field, SQLAlchemy
```

```
# =====  
# MODELO 1: CLIENTE  
# =====  
"""  
| Campo      | Tipo de Dato (SQL) | PK / FK | Nullable |  
| NIF        | `VARCHAR(9)`      | PK      | No       |
```

nombre	`VARCHAR(50)`		No
apellidos	`VARCHAR(50)`		No
direccion	`VARCHAR(100)`		Sí

```
"""
```

```
class Cliente(SQLModel, table=True):
    # NIF es la Clave Primaria (PK), tipo VARCHAR(9)
    NIF: str = Field(primary_key=True, max_length=9)
    nombre: str = Field(max_length=50)
    apellidos: str = Field(max_length=50)
    direccion: Optional[str] = Field(default=None, max_length=100)
```

```
    # Validador para el campo NIF
    @field_validator("NIF")
    @classmethod
    def validar_NIF(cls, valor: str) -> str:
        # Limpiamos posibles espacios en blanco
        valor = valor.strip()
```

```
    # Validamos formato básico antes del cálculo matemático para evitar
    errores de ejecución
    if len(valor) != 9 or not valor[:-1].isdigit() or not valor[-1].isalpha():
        raise ValueError("El NIF debe constar de 8 números seguidos de 1
        letra.")
```

```
    letras = "TRWAGMYFPDXBNJZSQVHLCKE"
    resto = int(valor[:-1]) % 23
```

```
    if letras[resto] != valor[-1].upper():
        raise ValueError(
            f"NIF inválido: la letra no coincide con el número. Se esperaba
            '{letras[resto]}', pero se recibió '{valor[-1]}'"
        )
```

```
    print(f"NIF válido: {valor[-1]} coincide con la letra '{letras[resto]}")
    return valor
```

```
    def __str__(self):
        return f"Cliente(NIF='{self.NIF}', nombre='{self.nombre}',
        apellidos='{self.apellidos}')
```

```
# =====
# MODELO 2: PEDIDO
# =====
"""
```

Campo	Tipo de Dato (SQL)	PK / FK	Nullable
-------	--------------------	---------	----------

id_pedido	INT	PK	No	
Autoincremental.				
NIF_cliente	VARCHAR(9)	FK	No	Ref. NIF de Cliente.
importe	DECIMAL(10, 2)		Sí	

```
"""
```

```
class Pedido(SQLModel, table=True):
    # id_pedido es la Clave Primaria (PK), Autoincremental e INT, uso la clave
    # "sa_column_kwarg" de SQLModel para indicar que debe ser autoincremental,
    # ademas con la clase Optional[int] indicamos que es un campo que se puede
    # omitir al crear un nuevo Pedido, ya que se generará automáticamente, y evitar
    # el error en el INSERT de la funcion insertar_datos_iniciales() del main.py
    id_pedido: Optional[int] = Field(
        default=None, primary_key=True, sa_column_kwarg={"autoincrement": True}
    )

    # NIF_cliente es Clave Foránea (FK) que referencia al campo NIF de la tabla
    # Cliente
    NIF_cliente: str = Field(foreign_key="cliente.NIF", max_length=9)

    # importe es un número DECIMAL(10,2) y puede ser nulo
    importe: Decimal = Field(default=None, decimal_places=2, max_digits=10)

    def __str__(self):
        return f"Pedido(id_pedido={self.id_pedido},
NIF_cliente='{self.NIF_cliente}', importe={self.importe})"
```

5. Se proporciona la lista de clientes:

clientes = [

 Cliente(NIF="12345678Z", nombre="Juan", apellidos="Pérez García", direccion="Calle Mayor 10, 28013 Madrid"),

 Cliente(NIF="87654321T", nombre="María", apellidos="López Rodríguez", direccion="Avenida de la Libertad 25, 08001 Barcelona"),

 Cliente(NIF="11223344K", nombre="Carlos", apellidos="Martín Sánchez", direccion="Plaza de España 5, 41001 Sevilla"),

 Cliente(NIF="99887766L", nombre="Ana", apellidos="Gómez Fernández", direccion="Calle Colón 12, 46004 Valencia"),

]

Y la lista de pedidos:

pedidos = [

 Pedido(NIF_cliente="99887766L", importe=Decimal("45.50")),

 Pedido(NIF_cliente="99887766L", importe=Decimal("120.00")),

 Pedido(NIF_cliente="99887766L", importe=Decimal("89.99")),

 Pedido(NIF_cliente="99887766L", importe=Decimal("15.75")),

 Pedido(NIF_cliente="99887766L", importe=Decimal("200.50")),

 Pedido(NIF_cliente="99887766L", importe=Decimal("55.20")),

 Pedido(NIF_cliente="12345678Z", importe=Decimal("30.00")),

 Pedido(NIF_cliente="12345678Z", importe=Decimal("42.50")),

 Pedido(NIF_cliente="12345678Z", importe=Decimal("18.90")),

 Pedido(NIF_cliente="87654321T", importe=Decimal("65.00")),

 Pedido(NIF_cliente="87654321T", importe=Decimal("99.95")),

 Pedido(NIF_cliente="87654321T", importe=Decimal("12.30")),

 Pedido(NIF_cliente="11223344K", importe=Decimal("210.00")),

 Pedido(NIF_cliente="11223344K", importe=Decimal("54.40")),

 Pedido(NIF_cliente="11223344K", importe=Decimal("33.15")),

]

- Habilitar los decoradores necesarios para el acceso (END POINTS) por navegador según la operación de que se trate (GET, POST, PUT, PATCH, DELETE, HEAD, etc)
- Hacer las inserciones vía ORM de los clientes y pedidos anteriores, por navegador.

```
# --- EJERCICIO 5.2: Inserciones vía ORM de los clientes y pedidos anteriores
---
```

```
@app.post("/insertar-datos/")
async def insertar_datos_iniciales(session: session_dep):
    # Comprobar si ya existen datos para evitar duplicados
    clientes_existentes = session.exec(select(Cliente)).first()
    if clientes_existentes:
        return {"mensaje": "Los datos iniciales ya fueron insertados
previamente."}
```

```
lista_clientes = [
    Cliente(
        NIF="12345678Z",
        nombre="Juan",
        apellidos="Pérez García",
        direccion="Calle Mayor 10, 28013 Madrid",
    ),
    Cliente(
        NIF="87654321T",
        nombre="María",
        apellidos="López Rodríguez",
        direccion="Avenida de la Libertad 25, 08001 Barcelona",
    ),
    Cliente(
        NIF="11223344K",
        nombre="Carlos",
        apellidos="Martín Sánchez",
        direccion="Plaza de España 5, 41001 Sevilla",
    ),
    Cliente(
        NIF="99887766L",
        nombre="Ana",
        apellidos="Gómez Fernández",
        direccion="Calle Colón 12, 46004 Valencia",
    ),
]
```

```
lista_pedidos = [
    Pedido(NIF_cliente="99887766L", importe=Decimal("45.50")),
    Pedido(NIF_cliente="99887766L", importe=Decimal("120.00")),
    Pedido(NIF_cliente="99887766L", importe=Decimal("89.99")),
    Pedido(NIF_cliente="99887766L", importe=Decimal("15.75")),
    Pedido(NIF_cliente="99887766L", importe=Decimal("200.50")),
]
```



```

Pedido(NIF_cliente="99887766L", importe=Decimal("55.20")),
Pedido(NIF_cliente="12345678Z", importe=Decimal("30.00")),
Pedido(NIF_cliente="12345678Z", importe=Decimal("42.50")),
Pedido(NIF_cliente="12345678Z", importe=Decimal("18.90")),
Pedido(NIF_cliente="87654321T", importe=Decimal("65.00")),
Pedido(NIF_cliente="87654321T", importe=Decimal("99.95")),
Pedido(NIF_cliente="87654321T", importe=Decimal("12.30")),
Pedido(NIF_cliente="11223344K", importe=Decimal("210.00")),
Pedido(NIF_cliente="11223344K", importe=Decimal("54.40")),
Pedido(NIF_cliente="11223344K", importe=Decimal("33.15")),
]

```

```

# Guardar en la Base de Datos con el ORM
for cliente in lista_clientes:
    session.add(cliente)

```

```

# Hacemos commit intermedio para que existan las PK de los clientes antes de
meter los pedidos (por la FK)
session.commit()

```

```

for pedido in lista_pedidos:
    session.add(pedido)

```

```

session.commit()
return {"mensaje": "Clientes y pedidos insertados correctamente mediante el
ORM."}

```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/insertar-datos/' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8080/insertar-datos/
```

Server response

Code

Details

200

Response body

```
{
  "mensaje": "Los datos iniciales ya fueron insertados previamente."
}
```

Download

Response headers

```
content-length: 67
content-type: application/json
date: Mon, 01 Jun 2026 11:24:07 GMT
server: uvicorn
```

Responses

- Hacer la selección vía ORM de todos los pedidos del cliente cuyo nombre es “Ana”, por navegador.

```
# --- EJERCICIO 5.3: Selección de todos los pedidos del cliente "Ana" ---
@app.get("/pedidos/cliente/ana/")
async def obtener_pedidos_ana(session: session_dep):
    # Unimos (JOIN) Pedido y Cliente, filtrando donde Cliente.nombre sea "Ana"
    declaracion = select(Pedido).join(Cliente).where(Cliente.nombre == "Ana")
    resultados = session.exec(declaracion).all()

    if not resultados:
        raise HTTPException(
            status_code=404, detail="No se encontraron pedidos para la cliente Ana."
        )
    return resultados
```

Request URL
http://127.0.0.1:8000/pedidos/cliente/ana/

Server response

Code	Details
200	Response body <pre>[{"id_pedido": 1, "importe": "45.50", "NIF_cliente": "99887766L"}, {"id_pedido": 2, "importe": "120.00", "NIF_cliente": "99887766L"}, {"id_pedido": 3, "importe": "89.99", "NIF_cliente": "99887766L"}, {"id_pedido": 4, "importe": "15.75", "NIF_cliente": "99887766L"}, {"id_pedido": 5, "importe": "200.50", "NIF_cliente": "99887766L"}, {"id_pedido": 6, "importe": "55.20", "NIF_cliente": "99887766L"}]</pre> Response headers <pre>content-length: 363 content-type: application/json date: Mon, 01 Jun 2026 11:26:08 GMT server: uvicorn</pre>

- Hacer la selección vía ORM de todos los pedidos que tienen importe entre 50 y 90, por navegador.

```
# --- EJERCICIO 5.4: Selección de pedidos con importe entre 50 y 90 ---

@app.get("/pedidos/rango-importe/")
async def obtener_pedidos_por_rango(session: SessionDep):
    # Filtrado por rango usando operadores relacionales comunes, evitando
    # conflictos de tipos
    declaracion = select(Pedido).where(
        Pedido.importe >= Decimal("50"), Pedido.importe <= 90
    )
    resultados = session.exec(declaracion).all()

    if not resultados:
        raise HTTPException(
            status_code=404,
            detail="No hay pedidos en el rango de importe especificado (50-90).",
        )
    return resultados
```

Request URL

http://127.0.0.1:8000/pedidos/rango-importe/

Server response

Code	Details
200	Response body <pre>[{ "id_pedido": 3, "importe": "89.99", "NIF_cliente": "99887766L" }, { "id_pedido": 6, "importe": "55.20", "NIF_cliente": "99887766L" }, { "id_pedido": 10, "importe": "65.00", "NIF_cliente": "87654321T" }, { "id_pedido": 14, "importe": "54.40", "NIF_cliente": "11223344K" }]</pre> Response headers <pre>content-length: 243 content-type: application/json date: Mon, 01 Jun 2026 11:27:48 GMT server: uvicorn</pre>

Download

- Actualizar vía ORM todos los pedidos del cliente con NIF “12345678Z” para que tengan un descuento del 10%, por navegador. (3 puntos).

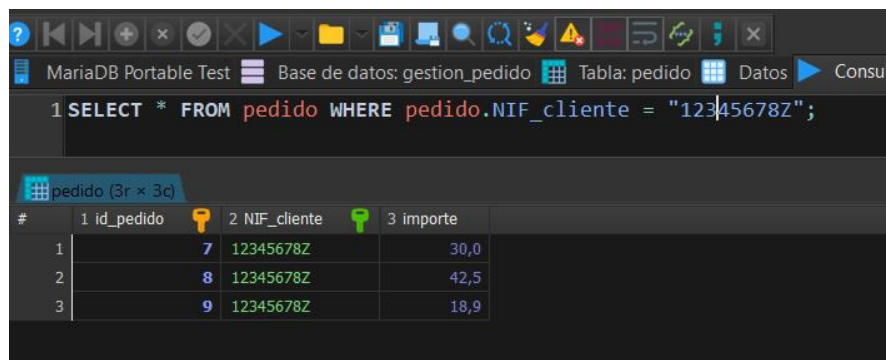
```
# --- EJERCICIO 5.5: Actualizar pedidos de "12345678Z" con un 10% de descuento
---
@app.put("/pedidos/aplicar-descuento-nif/")
async def aplicar_descuento_juan(session: session_dep):
    # Buscamos todos los pedidos asociados al NIF indicado
    declaracion = select(Pedido).where(Pedido.NIF_cliente == "12345678Z")
    pedidos_cliente = session.exec(declaracion).all()

    if not pedidos_cliente:
        raise HTTPException(
            status_code=404, detail="No se encontraron pedidos para el NIF
12345678Z."
        )

    # Modificamos el importe aplicando el 10% de descuento (multiplicar por
0.90)
    for pedido in pedidos_cliente:
        if pedido.importe is not None:
            # Convertimos el flotante a Decimal para mantener consistencia de tipos
            pedido.importe = pedido.importe * Decimal("0.90")
            session.add(pedido)

    session.commit()
    return {
        "mensaje": f"Se ha aplicado un 10% de descuento a los
{len(pedidos_cliente)} pedidos del NIF 12345678Z."
    }
```

PEDIDOS DEL NIF 12345678Z ANTES DEL CAMBIO, consulta realizada directamente en la Base de datos mediante SQL



The screenshot shows the MariaDB Portable Test interface. The top bar includes navigation icons and the text 'MariaDB Portable Test'. Below this, the database name 'Base de datos: gestion_pedido' and the table name 'Tabla: pedido' are displayed. The SQL query entered is '1 SELECT * FROM pedido WHERE pedido.NIF_cliente = "12345678Z";'. The results are shown in a table with 3 columns: '1 id_pedido', '2 NIF_cliente', and '3 importe'. The table contains 3 rows of data.

#	1 id_pedido	2 NIF_cliente	3 importe
1	7	12345678Z	30,0
2	8	12345678Z	42,5
3	9	12345678Z	18,9

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/pedidos/aplicar-descuento-nif/' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/pedidos/aplicar-descuento-nif/

Server response

Code Details

200

Response body

```
{
  "mensaje": "Se ha aplicado un 10% de descuento a los 3 pedidos del NIF 12345678Z."
}
```

Response headers

```
content-length: 83
content-type: application/json
date: Mon, 01 Jun 2026 11:36:31 GMT
server: uvicorn
```

Responses

PEDIDOS DEL NIF 12345678Z DESPUES DEL CAMBIO, consulta realizada directamente en la Base de datos mediante SQL

MariaDB Portable Test Base de datos: gestion_pedido Tabla: pedido Datos

```
1 SELECT * FROM pedido WHERE pedido.NIF_cliente = "12345678Z";
```

pedido (3r x 3c)

#	1 id_pedido	2 NIF_cliente	3 importe
1	7	12345678Z	27,0
2	8	12345678Z	38,25
3	9	12345678Z	17,01